Alan 6 – AI Systems / Efficiency

DistilBERT Duygu Analizi Modelinde Dynamic Quantization ve Batching ile Çıkarım Verimliliği:

Doğruluk – Gecikme Ödünleşiminin Deneysel Analizi

Özet

Bu çalışmada, önceden eğitilmiş bir metin sınıflandırma modelinin servis ortamındaki gecikmesini ve kaynak kullanımını, doğruluğu ciddi ölçüde düşürmeden azaltma olanakları deneysel olarak incelenmiştir. SST-2 veri seti için ince ayarlanmış DistilBERT modeline iki optimizasyon uygulanmıştır: yalnızca Linear katmanları hedefleyen dinamik INT8 quantization ve batching (batch boyutu 1, 8, 16 ve 32). Deney, iki model sürümü (FP32, INT8) ile dört batch boyutundan oluşan 2×4 faktöriyel bir tasarımla, CPU üzerinde, her hücre için 10 ısınma turu ve 100 ölçümden oluşan üç bağımsız ve rastgele sıralı çalıştırma ile yürütülmüştür. Doğruluk, SST-2 doğrulama setinin ilk 500 cümlesi üzerinde ölçülmüş; modeller arasındaki fark kesin McNemar testi ve eşleştirilmiş bootstrap güven aralığıyla değerlendirilmiştir.

Quantization, tek istekte (batch=1) medyan gecikmeyi 17,61 ms'den 10,59 ms'ye (1,66 kat), 95. yüzdelik gecikmeyi 19,84 ms'den 12,28 ms'ye düşürmüştür. Aynı batch boyutundaki hızlanma, daha büyük batch'lerde 2,68–2,98 kata çıkmıştır. Quantization ile batching birlikte, throughput'u FP32 tek istek düzeyindeki 56,4 örnek/s'den INT8 batch=16 düzeyinde 204,8 örnek/s'ye (3,6 kat) yükseltmiştir. Model dosyası %48,21 küçülmüştür. Doğruluk %91,2'den %90,4'e gerilemiştir; bu fark istatistiksel olarak anlamlı değildir ($p = 0,45$), ancak %95 güven aralığı [−2,4; +0,8] yüzde puan olduğundan, kaybın önceden belirlenen 1 puanlık toleransın altında olduğu da gösterilememiştir. Ayrıca INT8 modelin doğruluğunun batch boyutuna bağlı olarak değiştiği gözlenmiş ve bu durum dinamik aktivasyon ölçeklemesiyle açıklanmıştır.

Anahtar kelimeler: model verimliliği, dynamic quantization, batching, DistilBERT, çıkarım gecikmesi, McNemar testi

1. Giriş

Makine öğrenmesi modellerinin yaşam döngüsü iki aşamadan oluşur: modelin parametrelerini öğrendiği eğitim (training) aşaması ve eğitilmiş modelin yeni girdiler için tahmin ürettiği çıkarım (inference) aşaması. Eğitim genellikle bir kez ya da seyrek aralıklarla yapılırken, çıkarım modelin kullanıldığı süre boyunca her kullanıcı isteğinde tekrarlanır. Bu nedenle yoğun kullanılan sistemlerde toplam maliyetin ve kullanıcı deneyiminin belirleyicisi çoğunlukla çıkarım verimliliğidir.

Transformer tabanlı dil modelleri (Vaswani vd., 2017; Devlin vd., 2019) doğal dil işleme görevlerinde yüksek başarı sağlamakla birlikte, yüksek hesaplama ve bellek gereksinimleri nedeniyle servis ortamında gecikme ve maliyet sorunlarına yol açabilir. Bu sorunu azaltmak için önerilen yöntemler arasında bilgi damıtma (Sanh vd., 2019), quantization (Jacob vd., 2018; Gholami vd., 2021) ve isteklerin gruplanarak işlenmesi (batching) yer alır.

Bu rapor, zaten damıtılmış küçük bir model olan DistilBERT üzerinde dinamik quantization ve batching yöntemlerinin gecikme, throughput, model boyutu, bellek kullanımı ve doğruluk üzerindeki etkilerini, kontrollü ve tekrarlanabilir bir deney tasarımıyla incelemektedir.

2. Problem Tanımı

Görev tanımındaki senaryoya göre model, çevrimdışı (offline) testlerde yeterli doğruluğa ulaşmaktadır; ancak servis ortamında yanıt süresi ve maliyet kabul edilebilir sınırların üzerindedir. Problem, modeli yeniden eğitmeden, yalnızca çalıştırılma biçimini değiştirerek gecikmeyi ve kaynak kullanımını azaltmak ve bunu doğrulukta ciddi bir kayıp olmadan gerçekleştirmektir.

Bu problem, birden çok metrik arasında bir ödünleşim (trade-off) içerir: gecikmeyi azaltan bir yöntem doğruluğu bir miktar düşürebilir; throughput'u artıran bir yöntem tek bir isteğin gecikmesini artırabilir. Dolayısıyla çözüm tek bir "en hızlı" konfigürasyon değil, kullanım senaryosuna göre gerekçelendirilmiş bir seçimdir.

3. Projenin Amacı

Projenin amacı, DistilBERT modeli için doğruluk–gecikme ödünleşimini ölçmek ve farklı servis senaryoları için uygun konfigürasyonu belirlemektir. Bu amaç doğrultusunda aşağıdaki araştırma soruları ele alınmıştır:

1. Dinamik INT8 quantization, aynı batch boyutunda gecikmeyi ve throughput'u ne ölçüde değiştirir?
2. Batching, FP32 ve INT8 modellerin throughput'unu ve çağrı başına gecikmesini nasıl etkiler? İki optimizasyon arasında etkileşim var mıdır?
3. Quantization model dosya boyutunu ve çalışma belleğini ne kadar azaltır?
4. Quantization'ın neden olduğu doğruluk farkı istatistiksel olarak anlamlı ve pratik açıdan önemli midir?

4. Kullanılan Veri Seti: SST-2

Stanford Sentiment Treebank (SST), film eleştirilerinden alınan cümlelerin insanlar tarafından duygu açısından etiketlendiği bir derlemdir (Socher vd., 2013). İkili sürümü olan SST-2, cümleleri olumlu (1) ve olumsuz (0) olarak sınıflandırır ve GLUE karşılaştırma setinin görevlerinden biridir (Wang vd., 2019). Test

bölümünün etiketleri kamuya açık olmadığından, değerlendirme doğrulama (validation) bölümü üzerinde yapılmıştır. Kullanılan model bu cümleleri eğitimde görmediği için değerlendirme yansızdır.

Veri Hugging Face Datasets kütüphanesi ile yüklenmiş (Lhoest vd., 2021) ve doğrulama setinin ilk 500 örneği kullanılmıştır. Bu seçim deterministiktir ve sonuçların tekrar üretilmesine olanak verir. Örneklerin 265'i olumlu (%53,0), 235'i olumsuzdur (%47,0). Sınıfların dengeye yakın olması nedeniyle doğruluk (accuracy) birincil metrik olarak uygundur.

5. Kullanılan Model: DistilBERT

DistilBERT, BERT modelinden bilgi damıtma yöntemiyle elde edilen, 6 katmanlı daha küçük bir Transformer modelidir; yazarlar modelin BERT'in dil anlama performansının yaklaşık %97'sini koruyarak %40 daha küçük ve %60 daha hızlı olduğunu bildirmiştir (Sanh vd., 2019). Bu çalışmada Hugging Face'te yayımlanan ve SST-2 için ince ayarlanmış distilbert-base-uncased-finetuned-sst-2-english sürümü kullanılmıştır.

Notebook'ta yapılan parametre sayımına göre model toplam 66,96 milyon parametre içermektedir. Bunların 23,84 milyonu (%35,6) kelime ve konum gömme (embedding) tablolarında, 43,10 milyonu (%64,4) tam bağlantılı (Linear) katmanlarda, 0,02 milyonu ise LayerNorm gibi diğer katmanlardadır. Her Transformer katmanında altı Linear katman (q_lin, k_lin, v_lin, out_lin, lin1, lin2) bulunur; sınıflandırma başlığındaki iki katmanla birlikte toplam 38 Linear katman vardır. Bu dağılım, quantization sonrası model boyutunun yorumlanmasında belirleyicidir (Bölüm 13).

6. Deney Ortamı ve Kullanılan Teknolojiler

Tüm ölçümler aynı bilgisayarda ve aynı oturumda yapılmıştır. Notebook, deneyin başında ve sonunda donanım ve yazılım ortamının bir parmak izini çıkarmış; iki parmak izi arasında fark bulunmamıştır. Ortam bilgileri Tablo 1'de verilmiştir. Hesaplamalar yalnızca CPU üzerinde yapılmış, PyTorch iş parçacığı sayısı deney boyunca 8'de sabit tutulmuştur.

Tablo 1. Deney ortamı (environment.json)

Bileşen	Değer
İşletim sistemi	Windows 10
İşlemci	AMD64 Family 26 Model 96 (AuthenticAMD)
Çekirdek	8 fiziksel / 16 mantıksal; PyTorch iş parçacığı: 8
RAM	16,3 GB
Cihaz	CPU (GPU kullanılmadı)
Quantization motoru	oneDNN
Python / PyTorch	3.11.18 / 2.15.0+cpu
Transformers / Datasets	5.22.1 / 5.1.0
NumPy / pandas / psutil	2.4.6 / 3.1.0 / 7.2.2
Bellek ölçüm yöntemi	psutil memory_info().rss (Windows Working Set) ve peak_wset

Kullanılan başlıca kütüphaneler PyTorch (Paszke vd., 2019), Hugging Face Transformers (Wolf vd., 2020) ve Datasets'tir; tablo ve grafikler için pandas ve matplotlib, bellek ölçümü için psutil kullanılmıştır.

7. Baseline Performans Ölçümü

Baseline, hiçbir optimizasyon uygulanmamış FP32 PyTorch modelidir. Batch=1 ile 500 cümle üzerinde ölçülen doğruluk %91,20'dir (456/500); %95 Wilson güven aralığı [%88,39; %93,38]'dir (Wilson, 1927). Bu değer modelin bilinen yaklaşık %91'lik performansıyla uyumludur ve kurulumun doğruluğunu teyit eder.

Tablo 2. Baseline (FP32) karışıklık matrisi, batch=1

	Tahmin: olumsuz (0)	**Tahmin: olumlu (1)**
Gerçek: olumsuz (0)	208 (TN)	27 (FP)
Gerçek: olumlu (1)	17 (FN)	248 (TP)

Modelin 44 hatasının 27'si yanlış pozitiftir; olumlu sınıf için kesinlik %90,2, duyarlılık %93,6'dır. Baseline'ın gecikme ve throughput değerleri Tablo 3'te optimize edilmiş sürümlerle birlikte verilmiştir. Batch=1'de baseline'ın medyan gecikmesi 17,61 ms, 95. yüzdelik gecikmesi 19,84 ms, ortalaması 17,75 ms, çalıştırma içi standart sapması 1,31 ms ve throughput'u 56,4 örnek/s'dir.

8. Dynamic Quantization

Quantization, model parametrelerinin daha düşük bit genişliğinde temsil edilmesidir. Afin quantization'da gerçek değer x, 8 bitlik tam sayı q, ölçek s ve sıfır noktası z ile şu şekilde temsil edilir (Jacob vd., 2018):

$$q = round(x / s) + z, \qquad \hat{x} = s \cdot (q - z)$$

FP32 bir değer 4 bayt, INT8 bir değer 1 bayt yer kaplar. Daha az bit, daha küçük model, daha az bellek trafiği ve CPU'nun tam sayı vektör komutlarıyla daha hızlı matris çarpımı anlamına gelir; buna karşılık yuvarlama hatası (quantization hatası) ortaya çıkar.

Dinamik quantization'da ağırlıklar önceden INT8'e dönüştürülür; aktivasyonların ölçeği ise her çağrıda, o anki tensörün minimum ve maksimum değerinden hesaplanır. Bu nedenle kalibrasyon verisi gerekmez ve yöntem modeli yeniden eğitmeden uygulanabilir (post-training quantization). Bu çalışmada PyTorch'un quantize_dynamic fonksiyonu, orijinal modelin bir kopyasına {torch.nn.Linear} katman türü ve torch.qint8 veri tipiyle uygulanmıştır. Notebook, 38 Linear katmanın tamamının DynamicQuantizedLinear katmanına dönüştüğünü doğrulamıştır (qscheme: per_tensor_affine). Embedding tabloları, LayerNorm katmanları ve Linear katmanların sapma (bias) değerleri FP32 olarak kalmıştır.

Yöntemin tercih edilme nedenleri şunlardır: Transformer'larda hesaplamanın büyük kısmı Linear katmanlardaki matris çarpımlarından oluşur; yöntem tek satır kodla ve ek veri gerektirmeden uygulanabilir; PyTorch'ta CPU çıkarımı için doğrudan desteklenir.

9. Batching

Batching, birden çok girdinin tek bir çağrıda birlikte işlenmesidir. Batch içindeki cümleler en uzun cümlenin uzunluğuna kadar doldurulur (padding) ve dolgu konumları attention mask ile maskelenir. Batching, işlemcinin paralel vektör birimlerini daha verimli kullanarak ve çağrı başına sabit maliyetleri (tokenizer çağrısı, Python ek yükü) daha çok örneğe paylaştırarak throughput'u artırabilir. Buna karşılık bir batch'in işlenme süresi tek bir girdininkinden uzundur; gerçek bir serviste buna batch'in dolmasını bekleme süresi de eklenir. Bu nedenle batching toplam işi hızlandırırken tek bir isteğin gecikmesini artırabilir.

Bu çalışmada batch boyutları 1, 8, 16 ve 32 olarak belirlenmiştir. Gecikme bir batch'in tamamının işlenme süresi olarak tanımlanmış, ayrıca örnek başına süre (ortalama süre / batch boyutu) raporlanmıştır.

10. Deney Tasarımı ve Benchmark Metodolojisi

İki optimizasyonun etkilerini birbirinden ayırmak için 2×4 faktöriyel bir tasarım kullanılmıştır: model sürümü (FP32, INT8) × batch boyutu (1, 8, 16, 32). Aynı batch boyutundaki iki model arasındaki fark yalnızca quantization etkisini, aynı model içindeki batch boyutları arasındaki fark yalnızca batching etkisini verir.

- **Sabit girdi:** Seed = 42 ile 500 cümleden 32 cümle bir kez seçilmiş; batch=b için bu listenin ilk b cümlesi kullanılmıştır. Her tekrarda aynı girdi işlenmiştir.
- **Uçtan uca ölçüm:** Gecikme, tokenization ve model çalıştırmayı birlikte kapsar; time.perf_counter() ile milisaniye cinsinden ölçülmüştür.
- **Isınma ve tekrar:** Her ölçümden önce 10 ısınma turu yapılıp atılmış, ardından 100 ölçüm alınmıştır.
- **Bağımsız çalıştırmalar ve randomizasyon:** 8 hücrenin tamamı üç kez ölçülmüş; her çalıştırmada hücrelerin ölçüm sırası farklı bir seed ile rastgele karıştırılmıştır. Böylece zamanla değişen etkilerin (ısınma, arka plan işleri) sistematik olarak tek bir konfigürasyona yüklenmesi önlenmiştir.
- **Raporlanan istatistikler:** Medyan (p50), 95. yüzdelik (p95), ortalama, örneklem standart sapması (çalıştırma içi) ve throughput = (batch boyutu × 100) / toplam süre. Üç çalıştırmanın ortalaması ve çalıştırmalar arası standart sapması ayrıca verilmiştir.
- **Doğruluk:** Her hücre için 500 cümle üzerinde ölçülmüştür. Birincil karşılaştırma, önceden batch=1 olarak belirlenmiştir; diğer batch boyutları duyarlılık analizidir.
- **Bellek:** Her model, ayrı ve temiz bir Python sürecinde yüklenmiştir. Kütüphaneler yüklendikten sonra (RSS_o), model yüklendikten sonra (RSS_1) ve bir çıkarımdan sonra (RSS_2) süreç RSS'i okunmuş; tepe değer Windows'ta peak_wset alanından alınmıştır. Model kaynaklı bellek artışı $RSS_2 - RSS_o$ olarak tanımlanmıştır. Ayrıca modelin tensörlerinin bayt toplamı (tensör belleği) hesaplanmıştır.
- **İstatistiksel analiz:** Doğruluk farkı kesin McNemar testi, eşleştirilmiş bootstrap (10.000 tekrar) ve eşleştirilmiş Wald güven aralığıyla değerlendirilmiştir. Pratik önem için kabul edilebilir doğruluk kaybı önceden 1 yüzde puan olarak belirlenmiştir.

Notebook'ta Bölüm 8 ve 11'de her model için tek çalıştırmalık bir ön ölçüm de yapılmıştır. Bu raporda verilen tüm gecikme ve throughput değerleri, aksi belirtilmedikçe, üç bağımsız çalıştırmanın sonuçlarıdır. Ön ölçüm ile nihai ölçüm arasında küçük farklar vardır (örneğin batch=1'de ön ölçüm hızlanmayı 1,84 kat, nihai ölçüm 1,66 kat göstermektedir); nihai değerler esas alınmıştır.

11. Deneysel Sonuçlar

Tablo 3, 2×4 tasarımın sekiz hücresi için ana sonuçları özetlemektedir. Gecikme ve throughput değerleri üç çalıştırmanın ortalamasıdır; ± işaretinden sonraki değer çalıştırmalar arası standart sapmadır.

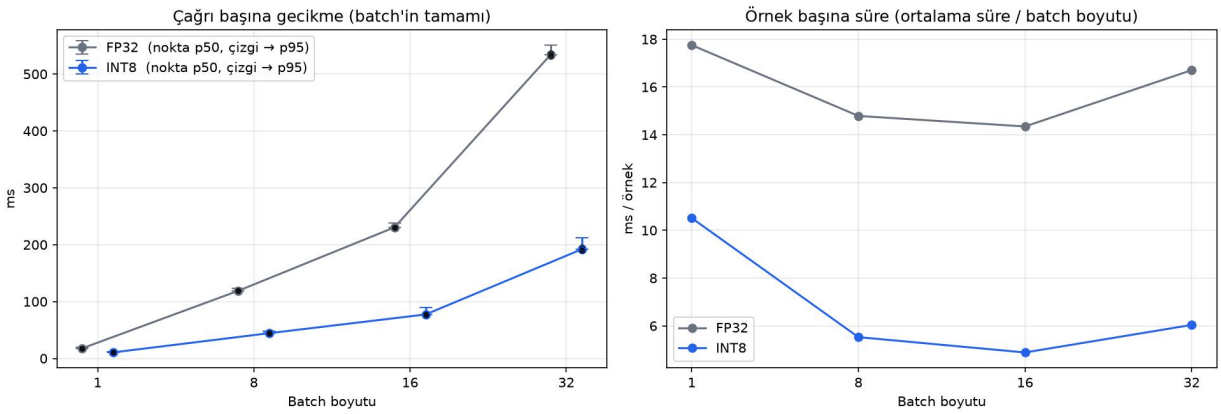
Tablo 3. Ana sonuçlar (results_table.csv)

Model	Batch	Doğruluk (%)	Δ Doğ. (puan)	P50 (ms)	P95 (ms)	Ort. (ms)	Std (ms)	Throughput (örnek/s)	Dosya (MB)
FP32	1	91,2	0,0	17,61 ± 0,36	19,84 ± 1,09	17,75	1,31	56,4 ± 1,6	267,86
FP32	8	91,2	0,0	118,86 ± 0,61	123,22 ± 0,41	118,30	4,15	67,6 ± 0,2	267,86
FP32	16	91,2	0,0	230,45 ± 0,35	237,53 ± 0,26	229,55	6,48	69,7 ± 0,1	267,86
FP32	32	91,2	0,0	534,13 ± 3,70	550,85 ± 5,48	534,56	16,55	59,9 ± 0,6	267,86
INT8	1	90,4	−0,8	10,59 ± 0,13	12,28 ± 0,45	10,51	1,30	95,2 ± 1,8	138,72
INT8	8	90,2	−1,0	44,41 ± 1,20	47,88 ± 1,05	44,22	2,81	181,0 ± 4,5	138,72
INT8	16	90,8	−0,4	77,33 ± 1,02	89,77 ± 15,09	78,16	4,81	204,8 ± 6,1	138,72
INT8	32	90,4	−0,8	192,02 ± 4,02	212,42 ± 18,51	193,20	10,76	165,7 ± 4,8	138,72

Not: Δ Doğruluk aynı batch boyutundaki FP32'ye göredir. Std, çalıştırma içi 100 ölçümün standart sapmasıdır (üç çalıştırma ortalaması).

11.1 Quantization etkisi

Aynı batch boyutunda INT8 model, FP32 modelden batch=1'de 1,66 kat, batch=8'de 2,68 kat, batch=16'da 2,98 kat ve batch=32'de 2,78 kat daha düşük medyan gecikme göstermiştir. P95 gecikmedeki azalma batch=1'de %38,1, diğer batch boyutlarında %61–62'dir. Hızlanmanın batch=1'de en düşük olması Amdahl yasasıyla tutarlıdır: tek bir cümlede tokenization, Python ek yükü, softmax, LayerNorm ve aktivasyonların her çağrıda yeniden ölçeklenmesi gibi quantization'ın hızlandırmadığı işlemler toplam sürenin daha büyük bir payını oluşturur. Bu açıklama bir hipotezdir; bu çalışmada süre dağılımı profil çıkarılarak doğrulanmamıştır.

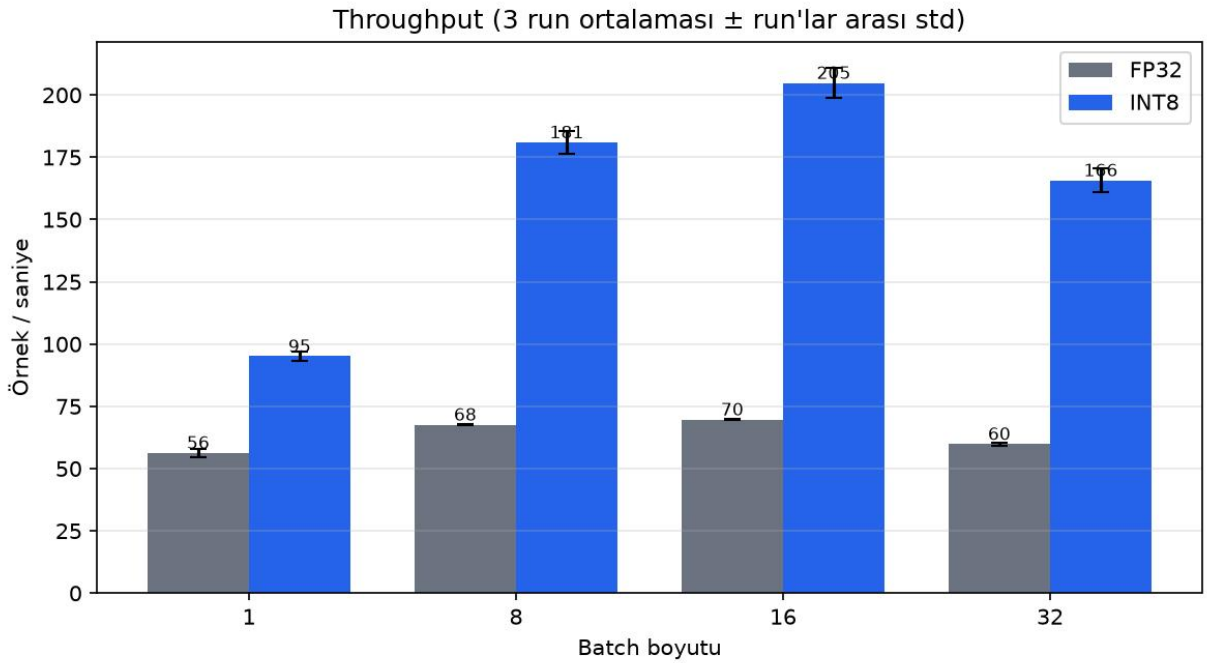


Şekil 1. Sol: çağrı başına gecikme (nokta p50, dikey çizgi p95'e kadar; küçük siyah noktalar tek tek çalıştırmaların p50 değerleri). Sağ: örnek başına süre (ortalama süre / batch boyutu).

11.2 Batching etkisi

Çağrı başına gecikme, batch boyutu arttıkça her iki modelde de artmıştır (Şekil 1, sol). Örnek başına süre ise U biçimli bir seyir izlemiş ve her iki modelde de en düşük değerine batch=16'da ulaşmıştır: FP32 için 14,35 ms/örnek, INT8 için 4,88 ms/örnek (Şekil 1, sağ). Batch=1'e göre throughput artışı FP32'de en fazla 1,24 kat (batch=16), INT8'de ise 2,15 kat (batch=16) olmuştur (Şekil 2). Bu, iki faktör arasında bir etkileşime işaret eder: batching, quantize edilmiş modele FP32 modelden belirgin biçimde daha fazla yarar sağlamaktadır. Olası açıklama, INT8 modelde matris çarpımları hızlandığı için sabit maliyetlerin toplam sürede daha büyük pay tutması ve batching'in bu maliyetleri paylaştırmasıdır.

Batch=32'de throughput her iki modelde de batch=16'ya göre düşmüştür (FP32: 69,7 → 59,9; INT8: 204,8 → 165,7 örnek/s). Olası nedenler, batch büyüdükçe en uzun cümlenin uzamasıyla artan dolgu (padding) hesabı, attention maliyetinin dizi uzunluğunun karesiyle büyümesi ve büyük aktivasyonların işlemci önbelleğine sığmamasıdır. Batch'ler iç içe seçildiğinden batch=32'nin en uzun cümlesi batch=16'nınkinden kısa olamaz. Bu nedenlerin katkıları ayrı ayrı ölçülmemiştir.



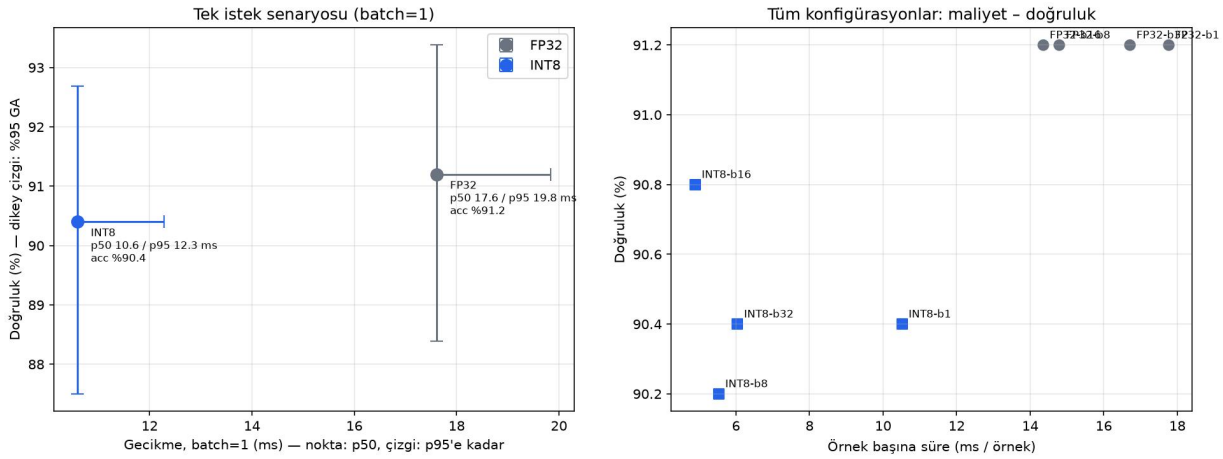
Şekil 2. Throughput (üç çalıştırma ortalaması ± çalıştırmalar arası standart sapma).

11.3 Ölçümlerin kararlılığı

Medyan gecikmenin çalıştırmalar arası değişim katsayısı tüm hücrelerde %3'ün altında kalmıştır (en yüksek: INT8 batch=8, %2,70). Throughput için de değişim katsayısı en fazla %3,0'dür. İstisna, INT8 batch=16 ve batch=32 hücrelerinin p95 değerleridir: birinci çalıştırmada bu iki hücre ilk ölçülen hücrelerdi ve p95 değerleri 107,16 ms ve 233,66 ms olarak ölçülmüştür; diğer iki çalıştırmada değerler 80,16–81,98 ms ve 199,75–203,83 ms aralığındadır. Aynı hücrelerin p50 değerleri çalıştırmalar arasında neredeyse değişmemiştir (76,19–78,16 ms). En olası açıklama, deney başlangıcında sistemin tam kararlı durumda olmamasıdır; bu yorum doğrulanmamıştır. Veriler dışlanmamış, etkisi çalıştırmalar arası standart sapmaya yansıtılmıştır. Ölçüm sırasının rastgeleleştirilmesi, bu tür bir başlangıç etkisinin her çalıştırmada aynı konfigürasyona denk gelmesini önlemiştir.

12. Accuracy–Latency Trade-off Analizi

Şekil 3'ün sol paneli, tek istek senaryosunda (batch=1) iki modelin konumunu göstermektedir. INT8 model yaklaşık 7 ms daha düşük medyan gecikmeye ve 0,8 puan daha düşük doğruluğa sahiptir; iki modelin %95 Wilson güven aralıkları ([%88,39; %93,38] ve [%87,50; %92,68]) büyük ölçüde örtüşmektedir. Sağ panel, sekiz konfigürasyonun örnek başına süre ve doğruluk değerlerini birlikte göstermektedir. Bu paneldeki dikey eksenin yalnızca %90,2–91,2 aralığını kapsadığına dikkat edilmelidir; INT8 konfigürasyonları arasındaki farklar en fazla 3 cümleye karşılık gelir ve bir konfigürasyonun diğerinden daha doğru olduğu sonucunu desteklemez.



Şekil 3. Doğruluk–gecikme ödünleşimi. Sol: batch=1 (yatay çizgi p50'den p95'e, dikey çizgi %95 Wilson güven aralığı). Sağ: tüm konfigürasyonlarda örnek başına süre ve doğruluk.

Konfigürasyon seçimi kullanım senaryosuna göre yapılmalıdır. Tablo 4, ölçülen sonuçlara dayanan önerileri özetlemektedir.

Tablo 4. Senaryoya göre önerilen konfigürasyon

Senaryo	Öncelik	Öneri	Gerekçe (ölçülen)
Gerçek zamanlı API, sohbet botu	Düşük p95	INT8, batch=1	p95 19,84 → 12,28 ms (%38 azalma)
Gece çalışan toplu etiketleme	Throughput, maliyet	INT8, batch=16	204,8 örnek/s; FP32 batch=1'in 3,6 katı
Doğruluğun kritik olduğu alan	Aynı çıktı, tekrarlanabilirlik	FP32	INT8'de 16 cümlenin tahmini değişti; INT8 doğruluğu batch'e bağlı
Kısıtlı RAM / disk	Bellek	INT8	Dosya 267,9 → 138,7 MB; RSS artışı 409,2 → 233,2 MB

Ölçülen sonuçlar ışığında genel amaçlı bir üretim ortamı için INT8 modelin tercih edilmesi uygun görünmektedir; ancak bu karar, doğruluk kaybının tolerans içinde olduğunun daha büyük bir değerlendirme setiyle gösterilmesine bağlı olmalıdır (Bölüm 14).

13. Model Boyutu ve Bellek Kullanımı Analizi

Model dosyası (state_dict) FP32'de 267,86 MB, INT8'de 138,72 MB'tır; azalma %48,21'dir. Tüm parametreler 4 bayttan 1 bayta inseydi azalma %75 olurdu. Azalmanın daha düşük kalmasının nedeni, parametrelerin %35,6'sını oluşturan embedding tablosunun FP32 kalmasıdır. Parametre dağılımından beklenen INT8 boyutu yaklaşık 23,84 M × 4 bayt + 43,10 M × 1 bayt + 0,02 M × 4 bayt ≈ 138,6 MB'tır; ölçülen değerle uyumludur. INT8 dosyasının yaklaşık %69'u artık küçülmeyen embedding tablosundan oluşmaktadır.

Bellek kullanımı dört ayrı kavramla raporlanmıştır (Tablo 5). Model dosya boyutu disk ve dağıtım maliyetini; tensör belleği modelin RAM'de tuttuğu sayıların kesin bayt toplamını; model kaynaklı RSS artışı modeli çalıştıran sürecin RAM ihtiyacındaki artışı; süreç tepe RSS ise kütüphaneler dahil sürecin ulaştığı en yüksek RAM kullanımını ifade eder.

Tablo 5. Model boyutu ve bellek kullanımı (MB)

Ölçüt	FP32	INT8	Azalma
Model dosya boyutu	267,86	138,72	%48,2
Tensör belleği	267,8	138,6	%48,2
Model kaynaklı RSS artışı (batch=1, çıkarım sonrası)	409,2	233,2	%43,0
Model kaynaklı RSS artışı (batch=32, çıkarım sonrası)	489,5	303,9	%37,9
Süreç tepe RSS (batch=1)	669,0	483,8	%27,7
Süreç tepe RSS (batch=32)	740,9	555,1	%25,1

Yalnızca kütüphanelerin yüklü olduğu süreçte RSS yaklaşık 251 MB'tır (dört ölçümde 250,7–251,3 MB). Bu sabit maliyet nedeniyle quantization'ın süreç toplamındaki göreli kazancı (%25–28), model düzeyindeki kazançtan (%43–48) düşüktür. Model kaynaklı RSS artışının tensör belleğinden büyük olması (FP32'de +141 MB, INT8'de +95 MB) bellek eşlemeli dosya sayfaları, bellek ayırıcının davranışı ve çalışma tamponlarıyla açıklanabilir; bu bileşenler ayrı ayrı ölçülmemiştir. Batch=32'nin getirdiği ek bellek iki modelde benzerdir (FP32: +80,3 MB; INT8: +70,7 MB), çünkü dinamik quantization'da aktivasyonlar FP32 olarak tutulur. Önceki çalışmada raporlanan 814 MB'lık değer de bir süreç toplamıdır ve model belleği olarak yorumlanmamalıdır.

14. İstatistiksel Değerlendirme

İki model aynı 500 cümle üzerinde değerlendirildiğinden gözlemler eşleştirilmiştir ve bağımsız iki oran testi uygun değildir. Eşleştirilmiş ikili sonuçlar için McNemar testi kullanılmıştır (McNemar, 1947; Dietterich, 1998). Test yalnızca uyuşmayan çiftlere dayanır; sıfır hipotezi altında $n_{10} \mid n \sim \text{Binom}(n, 0{,}5)$'tir. Uyuşmayan çift sayısı küçük olduğundan kesin (exact) binom sürümü kullanılmıştır:

$$p = \min[\,1\,;\; 2 \times \Sigma\, C(n, i) \times 0{,}5^{n}\,], \quad \text{toplam } i = 0, 1, \ldots, \min(n_{10}, n_{01}) \text{ üzerinden}$$

Tablo 6. Birincil karşılaştırma (batch=1) için 2×2 çapraz tablo

	INT8 doğru	INT8 yanlış	Toplam
FP32 doğru	446	10	456
FP32 yanlış	6	38	44
Toplam	452	48	500

Batch=1'de 16 uyuşmayan çift vardır ($n_{10} = 10$, $n_{01} = 6$); tahmin uyumu %96,8'dir. Kesin test $p = 2 \times 14893 / 65536 = 0{,}4545$ değerini verir; sıfır hipotezi reddedilmez. Doğruluk farkı $(n_{01} - n_{10}) / N = -0{,}8$ puandır. Farkın %95 güven aralığı eşleştirilmiş bootstrap ile [−2,4; +0,8] puan, eşleştirilmiş Wald yöntemiyle [−2,37; +0,77] puan olarak hesaplanmıştır (Efron, 1979). Tablo 7 tüm batch boyutlarının sonuçlarını göstermektedir.

Tablo 7. McNemar testi ve doğruluk farkının güven aralıkları (statistical_tests.csv)

Batch	Rol	FP32 (%)	INT8 (%)	Fark (puan)	n_{10} / n_{01}	McNemar p	Bootstrap %95 GA	Pratik karar
1	Birincil	91,2	90,4	−0,8	10 / 6	0,454	[−2,4; +0,8]	Belirsiz
8	Duyarlılık	91,2	90,2	−1,0	13 / 8	0,383	[−2,8; +0,8]	Belirsiz
16	Duyarlılık	91,2	90,8	−0,4	9 / 7	0,804	[−2,0; +1,2]	Belirsiz
32	Duyarlılık	91,2	90,4	−0,8	11 / 7	0,481	[−2,6; +0,8]	Belirsiz

İstatistiksel anlamlılık ile pratik önem birbirinden ayrılmalıdır. p değeri farkın tesadüfle açıklanıp açıklanamayacağını, güven aralığı ise farkın olası büyüklüğünü gösterir. Pratik önem için kabul edilebilir kayıp önceden 1 puan olarak belirlenmiş ve şu kural uygulanmıştır: güven aralığının alt sınırı −1 puanın üzerindeyse kayıp pratikte önemsiz, üst sınırı −1 puanın altındaysa pratikte önemli, aralık −1 puanı içeriyorsa sonuç belirsizdir. Birincil karşılaştırmada alt sınır −2,4 puan olduğundan sonuç belirsizdir. Dolayısıyla doğru yorum şudur: Doğruluk farkı istatistiksel olarak anlamlı değildir; ancak 500 örnekle kaybın 1 puanlık toleransın altında olduğu da gösterilememiştir. Uyuşmayan çift oranı (0,032) ve gözlenen fark sabit kalırsa, alt sınırın −1 puanın üzerine çıkması için kabaca 30.000 örnek gerekeceği hesaplanmıştır; bu, SST-2 doğrulama setinin büyüklüğünün (872) çok üzerindedir.

Duyarlılık analizinde FP32 modelin tahminleri dört batch boyutunda birebir aynı kalmış, INT8 modelin doğruluğu ise %90,2 ile %90,8 arasında değişmiştir. Bu durum, dinamik quantization'da aktivasyon ölçeğinin tensörün tamamı, yani batch'teki tüm cümleler üzerinden hesaplanmasının beklenen bir sonucudur. Dört test birlikte yapıldığından çoklu karşılaştırma riski bulunmaktadır; bu nedenle karar yalnızca önceden belirlenen birincil teste dayandırılmıştır. Kesin McNemar testinin konservatif olduğu bilinmektedir (Fagerland vd., 2013); mid-p sürümü bu veride yaklaşık 0,33 değerini verir ve sonucu değiştirmez.

Gecikme farkları için ayrı bir hipotez testi yapılmamıştır. Batch=1'de FP32 ve INT8 modellerin çalıştırma bazındaki p50 değerleri (17,24–17,96 ms ve 10,47–10,72 ms) hiç örtüşmemekte ve aradaki yaklaşık 7 ms'lik fark çalıştırmalar arası standart sapmanın (0,13–0,36 ms) yaklaşık 20 katıdır.

15. Sonuç ve Tartışma

Dinamik INT8 quantization, DistilBERT modelinin CPU üzerindeki gecikmesini tüm batch boyutlarında azaltmış, model boyutunu yaklaşık yarıya indirmiş ve batching ile birlikte kullanıldığında throughput'u 3,6 kat artırmıştır. Kazancın büyüklüğü koşula bağlıdır: hızlanma tek istekte 1,66 kat, büyük batch'lerde yaklaşık 3 kattır; bellek kazancı ise hangi ölçütün kullanıldığına göre %25 ile %48 arasında raporlanabilmektedir. Bu nedenle verimlilik sonuçlarının ölçüm koşulları ve kullanılan tanımlarla birlikte raporlanması gerekir.

Doğruluk açısından sonuç belirsizdir: kayıp istatistiksel olarak anlamlı değildir, ancak 1 puanlık toleransın altında olduğu da gösterilememiştir. Ayrıca quantize edilmiş modelin çıktılarının batch kompozisyonuna bağlı olması, dinamik batching kullanan sistemlerde tekrarlanabilirlik açısından dikkate alınması gereken bir bulgudur.

Önceki çalışmayla karşılaştırma. Kodu mevcut olmayan önceki bir çalışmada raporlanan değerler, bu çalışmanın batch=16 sonuçlarıyla büyük ölçüde örtüşmektedir: FP32 p50 238,55 ms'ye karşı 230,45 ms; INT8 p50 79,14 ms'ye karşı 77,33 ms; INT8 throughput 200,95'e karşı 204,84 örnek/s; doğruluklar ise birebir aynıdır (%91,2 ve %90,8). Önceki çalışmada throughput × ortalama gecikme çarpımının 16,0 örnek vermesiyle birlikte bu örtüşme, önceki "tekil" ölçümlerin batch=16 ile yapıldığı çıkarımını güçlü biçimde desteklemektedir; ancak eski benchmark kodu bulunmadığından bu çıkarım doğrulanmamış olarak kabul edilmiştir. Önceki çalışmadaki model boyutları (255,43 ve 132,29) MiB birimindedir ve bu çalışmadaki MB değerleriyle (267,86 MB = 255,45 MiB; 138,72 MB = 132,29 MiB) tutarlıdır. Önceki batch=8 özetindeki −1,00 puanlık doğruluk farkı da bu çalışmada aynen yeniden üretilmiştir; böylece önceki çalışmadaki −0,4 ve −1,0 puanlık farklı değerler, iki farklı batch boyutunda ölçülmüş olmalarıyla açıklanmıştır.

15.1 Yorum

Beklenti. Dinamik quantization'ın CPU'da gecikmeyi yaklaşık 2–3 kat azaltmasını, model boyutunu %75 küçültmesini ve doğruluğu 1 puandan az düşürmesini; batching'in ise örnek başına maliyeti düşürerek throughput'u artırmasını bekledim.

Sonuç. Batch=1'de quantization p50 gecikmeyi 17,61 ms'den 10,59 ms'ye, p95'i 19,84 ms'den 12,28 ms'ye indirdi (1,66 kat); daha büyük batch'lerde hızlanma 2,68–2,98 kat oldu. INT8 batch=16, FP32 batch=1'e göre 3,6 kat throughput sağladı (204,8 örnek/s). Model %48 küçüldü. Doğruluk %91,2'den %90,4'e indi; fark anlamlı değil (McNemar p = 0,45) ancak güven aralığı [−2,4; +0,8] puan olduğu için sonuç belirsiz.

Farkın olası nedenleri. Tek istekte quantization'ın hızlandırmadığı tokenization ve Python ek yükü toplam sürede büyük pay tutuyor (Amdahl yasası). Boyutun %75 değil %48 azalmasının nedeni, parametrelerin %36'sını oluşturan embedding tablosunun FP32 kalması. Batch=32'nin batch=16'dan yavaş olması muhtemelen dolgu ve önbellek etkilerinden kaynaklanıyor. INT8 doğruluğunun batch'e göre değişmesi dinamik aktivasyon ölçeklemesinin sonucu.

Sonraki adım. Doğruluk belirsizliğini azaltmak için 872 örneğin tamamı ve ek veri setleriyle değerlendirme; profil çıkararak darboğazın doğrulanması; uzunluğa göre gruplama, statik quantization ve ONNX Runtime denemeleri; modelin bir API arkasında eş zamanlı istek yüküyle test edilmesi.

16. Sınırlılıklar

- Ölçümler tek bir bilgisayarda yapılmıştır; INT8 hızlanması işlemcinin komut desteğine bağlı olduğundan sonuçlar başka donanımlara doğrudan genellenemez.
- Doğruluk 500 örnekle ölçülmüştür; güven aralıkları yaklaşık ±2–3 puandır ve 1 puanlık bir kaybı dışlamaya yetmemektedir.
- Gecikme, seed ile seçilen sabit 32 cümlelik bir havuzla ölçülmüştür; gerçek trafikteki cümle uzunluğu dağılımı farklı olabilir.
- Üç çalıştırma aynı Python sürecinde yapılmıştır ve tam bağımsız değildir.
- İstekler sıralı işlenmiştir; eş zamanlı istekler, kuyrukta bekleme ve batch'in dolmasını bekleme süresi ölçülmemiştir.
- Süre dağılımı profil çıkarılarak ölçülmemiştir; Amdahl yasasına dayalı açıklama ve batch=32'ye ilişkin açıklamalar hipotez niteliğindedir.
- Model kaynaklı RSS artışı işletim sistemi ve bellek ayırıcı davranışından etkilenir; bu nedenle tensör belleğiyle birlikte raporlanmıştır.

17. Gelecek Çalışmalar

1. SST-2 doğrulama setinin tamamı ve ek duygu analizi veri setleriyle doğruluk farkının daha dar güven aralıklarıyla değerlendirilmesi.
2. PyTorch Profiler ile sürenin tokenization, Linear katmanlar ve diğer işlemler arasındaki dağılımının ölçülmesi.
3. Cümleleri uzunluğa göre gruplama (length bucketing) ile dolgunun azaltılması ve batching kazancının yeniden ölçülmesi.

4. Kalibrasyonlu statik quantization, ONNX Runtime ve embedding katmanı için yalnızca ağırlık (weight-only) quantization yöntemlerinin aynı deney çerçevesinde karşılaştırılması.
5. Modelin bir API arkasına yerleştirilip eş zamanlı istek yükü altında, dinamik batching ile p95 gecikmenin istek hızının fonksiyonu olarak ölçülmesi.
6. Farklı CPU mimarilerinde ve GPU üzerinde (FP16/INT8) sonuçların tekrarlanması.

Kaynakça

Dean, J., & Barroso, L. A. (2013). The tail at scale. Communications of the ACM, 56(2), 74–80. https://doi.org/10.1145/2408776.2408794

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. Proceedings of NAACL-HLT 2019. https://arxiv.org/abs/1810.04805

Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. Neural Computation, 10(7), 1895–1923. https://doi.org/10.1162/089976698300017197

Efron, B. (1979). Bootstrap methods: Another look at the jackknife. The Annals of Statistics, 7(1), 1–26. https://doi.org/10.1214/aos/1176344552

Fagerland, M. W., Lydersen, S., & Laake, P. (2013). The McNemar test for binary matched-pairs data: Mid-p and asymptotic are better than exact conditional. BMC Medical Research Methodology, 13, 91. https://doi.org/10.1186/1471-2288-13-91

Gholami, A., Kim, S., Dong, Z., Yao, Z., Mahoney, M. W., & Keutzer, K. (2021). A survey of quantization methods for efficient neural network inference. arXiv:2103.13630. https://arxiv.org/abs/2103.13630

Hugging Face. (t.y.). distilbert-base-uncased-finetuned-sst-2-english [Model kartı]. https://huggingface.co/distilbert/distilbert-base-uncased-finetuned-sst-2-english

Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., & Kalenichenko, D. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. Proceedings of CVPR 2018. https://arxiv.org/abs/1712.05877

Lhoest, Q., vd. (2021). Datasets: A community library for natural language processing. Proceedings of EMNLP 2021: System Demonstrations. https://aclanthology.org/2021.emnlp-demo.21/

McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika, 12(2), 153–157. https://doi.org/10.1007/BF02295996

Paszke, A., vd. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in NeurIPS 32. https://arxiv.org/abs/1912.01703

PyTorch. (t.y.). (Dynamic) quantization on BERT [Eğitim belgesi]. https://pytorch.org/tutorials/intermediate/dynamic_quantization_bert_tutorial.html

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. arXiv:1910.01108. https://arxiv.org/abs/1910.01108

Socher, R., Perelygin, A., Wu, J., Chuang, J., Manning, C. D., Ng, A., & Potts, C. (2013). Recursive deep models for semantic compositionality over a sentiment treebank. Proceedings of EMNLP 2013, 1631–1642. https://aclanthology.org/D13-1170/

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. Advances in NeurIPS 30. https://arxiv.org/abs/1706.03762

Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., & Bowman, S. R. (2019). GLUE: A multi-task benchmark and analysis platform for natural language understanding. Proceedings of ICLR 2019. https://arxiv.org/abs/1804.07461

Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. Journal of the American Statistical Association, 22(158), 209–212. https://doi.org/10.1080/01621459.1927.10502953

Wolf, T., vd. (2020). Transformers: State-of-the-art natural language processing. Proceedings of EMNLP 2020: System Demonstrations, 38–45. https://aclanthology.org/2020.emnlp-demos.6/